MOBILE ENGINEERING HANDOFF

Cashfree UPI Intent in Native Android and iOS WebViews

Implementation contract for the RealFrnd Cashfree JS SDK checkout. This document tells the mobile teams exactly what to integrate, what the backend owns, which URLs and APIs are involved, and how to validate the complete UPI Intent journey.

Web checkout	Deployed and opening Cashfree sandbox checkout
Current endpoint	http://13.127.207.172:5052
Mobile platforms	Native Android and native iOS
Integration mode	Cashfree JS SDK inside app WebView
Recommended path	Cashfree feature flag plus native UPI URL interception
Scope	Pay-in only. Payout is a separate server-to-server product.

Temporary endpoint	The current IP endpoint is plain HTTP and is marked Not Secure. It is acceptable only for short sandbox diagnostics. Before mobile acceptance or production, move the same application behind an HTTPS domain and update APP_BASE_URL.

Security action	Cashfree credentials were exposed during setup and must be rotated. Never provide Cashfree client IDs, secrets, payout credentials, or webhook secrets to the mobile application or mobile developers.

Document owner: Web/Backend team | Consumers: Android, iOS, QA, DevOps, Cashfree account owner

1. Executive summary

The web application already creates Cashfree Payment Gateway orders, receives a payment_session_id, opens Cashfree JS SDK v3, handles the return URL, and verifies order status from the backend. The mobile application must not recreate those operations. Its responsibility is to host the payment site in a trusted WebView and redirect UPI schemes to installed payment apps.

What mobile receives A single payment website URL, initially http://13.127.207.172:5052 for sandbox diagnostics and later an HTTPS domain. The mobile binary receives no Cashfree API secrets.

Mandatory distinction

Flow	Direction	Does UPI Intent apply?	Owner
Pay-in	Customer pays RealFrnd	Yes	Web/backend + native WebView
UPI ID payout	RealFrnd pays beneficiary	No	Backend Payout API only
Bank payout	RealFrnd pays bank account	No	Backend Payout API only

Recommended solution

- Use the Cashfree feature-flag solution documented for JS SDK checkout inside WebViews.
- Implement native URL interception on Android and iOS before requesting the feature flag.
- Do not register the code-based JS bridge unless Cashfree specifically directs the team to use that alternative.
- Treat the backend Cashfree order status as authoritative; never trust the UPI app return alone.

Useful links

Resource	Clickable URL
Source repository	https://github.com/Abhishek-Real11/cashfree-payin-payout-poc
Existing mobile handoff notes	https://github.com/Abhishek-Real11/cashfree-payin-payout-poc/blob/main/MOBILE_UPI_HANDOFF.md
Live sandbox POC	http://13.127.207.172:5052
Backend config health endpoint	http://13.127.207.172:5052/api/config
Cashfree UPI Intent JS SDK guide	https://www.cashfree.com/docs/payments/online/mobile/misc/upi_intent_support_js_sdk
Cashfree Android sample	https://github.com/cashfree/CFWebIntegrationSample/blob/245b0d655475ad5f5595437efc1e6eb503774f68/app/src/main/java/com/cashfree/webintegration/MainActivity.kt
Cashfree iOS guidance	https://www.cashfree.com/docs/payments/online/web/custom-checkout-ios
Cashfree support form	https://merchant.cashfree.com/merchants/landing?env=prod&raise_issue=1
Cashfree checkout example	https://www.cashfree.com/devstudio/preview/pg/web/inlineCheckout
Cashfree webhook verification	https://www.cashfree.com/devstudio/preview/pg/tools/webhookVerification

2. End-to-end architecture and ownership

Step	Actor	Action	Result
1	Native app	Open payment website in WebView	Customer sees RealFrnd pay-in form
2	Web frontend	POST /api/payin/orders	Backend receives amount and customer details
3	Backend	Call Cashfree Create Order using server credentials	Cashfree returns payment_session_id
4	Web frontend	Call cashfree.checkout(...) with the session ID	Cashfree hosted checkout opens
5	Customer	Select a UPI app	Cashfree navigates to a UPI or intent URL
6	Native WebView	Intercept the UPI URL, open native app, cancel WebView navigation	GPay, PhonePe, Paytm, etc. opens
7	Customer	Approve or cancel in UPI app	User returns to existing app/WebView
8	Cashfree/Web	Return to APP_BASE_URL with order_id	Web requests server verification
9	Backend	GET Cashfree order status	Only PAID is treated as final success
10	Cashfree	Send signed webhook asynchronously	Backend reconciles final state

Responsibility matrix

Team	Must deliver	Must not do
Web/Backend	Order creation, session ID, return URL, server verification, webhook verification	Expose Cashfree secrets in browser or app
Android	Trusted WebView, scheme interception, Intent launch, fallback UX, Manifest queries	Load UPI URLs as web pages
iOS	Trusted WKWebView, scheme interception, UIApplication open, fallback UX, Info.plist schemes	Assume canOpenURL works without plist entries
DevOps	HTTPS domain, Nginx, PM2, protected environment file, logs, restricted port	Expose unauthenticated payout APIs publicly
Cashfree owner	Enable feature flag after mobile work is ready	Enable early and break checkout
QA	Physical-device matrix and final server-status verification	Declare success from app callback alone

3. What the mobile teams receive

Primary input	PAYMENT_WEB_URL = http://13.127.207.172:5052 for the current sandbox diagnostic deployment. Replace this with the final HTTPS domain before acceptance.

No direct Cashfree API integration is required in the mobile app

In the current design, Android and iOS load the web application. The web application calls the RealFrnd backend, and the backend calls Cashfree. The mobile team does not call Cashfree Create Order directly and does not receive payment_session_id as an application secret.

Backend endpoints used by the web layer

Method and path	Caller	Purpose	Mobile action
GET /api/config	Web	Environment and readiness booleans	None
POST /api/payin/orders	Web	Create Cashfree pay-in order	None
GET /api/payin/orders/{orderId}	Web	Fetch authoritative order status	None
POST /api/webhooks/payment	Cashfree	Signed payment event	None

Create order request

```
POST /api/payin/orders
Content-Type: application/json

{
  "amount": 10,
  "phone": "9999999999",
  "email": "customer@example.com"
}
```

Create order response

```
{
  "orderId": "poc_...",
  "paymentSessionId": "session_...",
  "orderStatus": "ACTIVE"
}
```

Status rule

Cashfree order status	Mobile/web interpretation
PAID	Payment success. This is the only final success state used by this POC.
ACTIVE	Order exists but payment is not complete. Keep pending/retry UI.
Any other value	Do not show success. Render the server response or mapped failure/pending state.

Never ship secrets	Do not send CASHFREE_CLIENT_ID, CASHFREE_CLIENT_SECRET, payout credentials, webhook signing material, or the server .env file to Android/iOS. Those values remain server-only.

4. Native Android implementation

Target behavior

When Cashfree navigates to upi://, intent://, phonepe://, tez:// or another supported PSP scheme, Android must open a native application and return true from shouldOverrideUrlLoading so the WebView does not attempt to render the URI.

1. **Load only the approved payment origin**
   Load http://13.127.207.172:5052 for current sandbox diagnostics. Replace with the final HTTPS origin and restrict navigation policy accordingly.
2. **Enable required WebView capabilities**
   Enable JavaScript and DOM storage. Disable file and content access unless another reviewed app feature requires them.
3. **Add Android 11+ package visibility**
   Declare queryable UPI schemes in AndroidManifest.xml. Place queries directly below manifest, not inside application.
4. **Intercept Cashfree UPI navigation**
   Handle both WebResourceRequest and the legacy String overload if the app supports older Android versions.
5. **Open a browsable Intent**
   For intent:// use Intent.parseUri. For direct schemes use ACTION_VIEW. Resolve before launching and display a recoverable no-app message.
6. **Preserve the WebView**
   Do not recreate or reload checkout when the UPI app opens. The customer must return to the same checkout state.

AndroidManifest.xml

```
<manifest ...>
  <uses-permission android:name="android.permission.INTERNET" />

  <queries>
    <intent>
      <action android:name="android.intent.action.VIEW" />
      <data android:scheme="upi" />
    </intent>
    <intent>
      <action android:name="android.intent.action.VIEW" />
      <data android:scheme="tez" />
    </intent>
    <intent>
      <action android:name="android.intent.action.VIEW" />
      <data android:scheme="gpay" />
    </intent>
    <intent>
      <action android:name="android.intent.action.VIEW" />
      <data android:scheme="phonepe" />
    </intent>
    <intent>
      <action android:name="android.intent.action.VIEW" />
      <data android:scheme="paytmmp" />
    </intent>
    <intent>
      <action android:name="android.intent.action.VIEW" />
      <data android:scheme="credpay" />
    </intent>
  </queries>

  <application ... />
</manifest>
```

Android WebView client - Kotlin reference

```
private val upiSchemes = setOf(
    "upi", "tez", "gpay", "paytmmp", "phonepe",
    "amazonpay", "credpay", "bhim", "navipay",
    "mobikwik", "myairtel", "popclubapp", "super",
    "kiwi", "simplypayupi", "whatsapp-consumer",
    "tnupi", "postpe", "imobile", "payzapp",
    "fpupi", "snapmint"
)

private fun openUpiUrl(view: WebView, rawUrl: String): Boolean {
    val uri = Uri.parse(rawUrl)
    val scheme = uri.scheme?.lowercase() ?: return false

    if (scheme == "intent") {
        return try {
            val intent = Intent.parseUri(
                rawUrl,
                Intent.URI_INTENT_SCHEME
            ).apply {
                addCategory(Intent.CATEGORY_BROWSABLE)
                component = null
                selector = null
            }

            if (intent.resolveActivity(packageManager) != null) {
                startActivity(intent)
            } else {
                Toast.makeText(
                    this,
                    "No supported UPI app is installed",
                    Toast.LENGTH_SHORT
                ).show()
            }
            true
        } catch (error: Exception) {
            Log.e("CashfreeUPI", "Invalid intent URL", error)
            true
        }
    }

    if (scheme in upiSchemes) {
        val intent = Intent(Intent.ACTION_VIEW, uri).apply {
            addCategory(Intent.CATEGORY_BROWSABLE)
        }
        if (intent.resolveActivity(packageManager) != null) {
            startActivity(intent)
        } else {
            Toast.makeText(
                this,
                "No supported UPI app is installed",
                Toast.LENGTH_SHORT
            ).show()
        }
        return true
    }

    return false
}

webView.settings.javaScriptEnabled = true
webView.settings.domStorageEnabled = true
webView.settings.allowFileAccess = false
webView.settings.allowContentAccess = false

webView.webViewClient = object : WebViewClient() {
    override fun shouldOverrideUrlLoading(
        view: WebView,
        request: WebResourceRequest
    ): Boolean = openUpiUrl(view, request.url.toString())

    @Suppress("DEPRECATION")
    override fun shouldOverrideUrlLoading(
        view: WebView,
        url: String
    ): Boolean = openUpiUrl(view, url)
}
```

Android security	Intercept only the documented UPI/intent schemes in the checkout context. Do not turn the WebView into a generic arbitrary-scheme launcher. Keep Safe Browsing enabled and allow HTTP/HTTPS navigation only to reviewed origins.

5. Native iOS implementation

Target behavior

WKWebView must detect a supported UPI scheme in decidePolicyFor, use canOpenURL/open to hand the request to an installed PSP application, cancel WebView navigation, and preserve the existing checkout instance for the return journey.

1. **Load only the approved payment origin**
   Load http://13.127.207.172:5052 for the temporary sandbox deployment, then replace it with HTTPS before acceptance.
2. **Declare every queried scheme**
   Add schemes to LSApplicationQueriesSchemes. Without this, canOpenURL can return false even when the app is installed.
3. **Implement WKNavigationDelegate**
   Intercept only supported UPI schemes. For all other navigation, call decisionHandler(.allow).
4. **Open externally and cancel WebView**
   Call UIApplication.shared.open and then decisionHandler(.cancel).
5. **Preserve checkout state**
   Do not destroy or reload WKWebView when the application resigns active. Let the customer return to the existing page.

Info.plist

```
<key>LSApplicationQueriesSchemes</key>
<array>
  <string>upi</string>
  <string>tez</string>
  <string>gpay</string>
  <string>paytmmp</string>
  <string>phonepe</string>
  <string>amazonpay</string>
  <string>credpay</string>
  <string>bhim</string>
  <string>navipay</string>
  <string>mobikwik</string>
  <string>myairtel</string>
  <string>popclubapp</string>
  <string>super</string>
  <string>kiwi</string>
  <string>simplypayupi</string>
  <string>whatsapp-consumer</string>
  <string>tnupi</string>
  <string>postpe</string>
  <string>imobile</string>
  <string>payzapp</string>
  <string>fpupi</string>
  <string>snapmint</string>
</array>
```

iOS WKNavigationDelegate - Swift reference

```
private let upiSchemes: Set<String> = [
    "upi", "tez", "gpay", "paytmmp", "phonepe",
    "amazonpay", "credpay", "bhim", "navipay",
    "mobikwik", "myairtel", "popclubapp", "super",
    "kiwi", "simplypayupi", "whatsapp-consumer",
    "tnupi", "postpe", "imobile", "payzapp",
    "fpupi", "snapmint"
]

func webView(
    _ webView: WKWebView,
    decidePolicyFor navigationAction: WKNavigationAction,
    decisionHandler: @escaping (WKNavigationActionPolicy) -> Void
) {
    guard
        let url = navigationAction.request.url,
        let scheme = url.scheme?.lowercased()
    else {
        decisionHandler(.allow)
        return
    }

    guard upiSchemes.contains(scheme) else {
        decisionHandler(.allow)
        return
    }

    if UIApplication.shared.canOpenURL(url) {
        UIApplication.shared.open(
            url,
            options: [:],
            completionHandler: nil
        )
    } else {
        // Present a recoverable "UPI app not installed" message.
    }

    decisionHandler(.cancel)
}
```

iOS security	Allow only documented schemes and trusted HTTP/HTTPS origins. Never invoke decisionHandler more than once. Keep the list synchronized with Cashfree's current official documentation and test each installed PSP on physical devices.

WebView lifecycle rule for both platforms

- Opening a UPI app must not create a second checkout order.
- Returning to the app must reveal the same WebView and order.
- Do not show success based on Android Intent extras or iOS application return alone.
- Wait for the web flow to return or invoke the backend order-status check.

6. Scheme coverage and Cashfree feature flag

Scheme	Typical app/use	Platform note
upi	Default UPI payment URI	Android and iOS
intent	Android intent URI	Android only
tez / gpay	Google Pay	Platform support varies
paytmmp	Paytm	Android and iOS where installed
phonepe	PhonePe	Android and iOS where installed
amazonpay	Amazon Pay	Where supported
credpay	CRED	Where supported
bhim	BHIM	Where supported
navipay	Navi	Where supported
mobikwik	MobiKwik	Where supported
myairtel	Airtel	Where supported
popclubapp	POP	Where supported
super	super.money	Where supported
kiwi	Kiwi	Where supported
simplypayupi	SimplyPay	Where supported
whatsapp-consumer	WhatsApp	Where supported
tnupi	Tata Neu	Where supported
postpe	BharatPe/PostPe	Where supported
imobile	iMobile	Where supported
payzapp	PayZapp	Where supported
fpupi	FamPay	Where supported
snapmint	Snapmint	Where supported

Feature-flag sequence

1. **Complete native handling first**
   Both Android and iOS builds must open UPI apps and handle no-app fallback.
2. **Run physical-device smoke tests**
   Validate schemes with SIM-bound UPI apps using the sandbox merchant.
3. **Request Cashfree activation**
   Raise a request using the Cashfree support form for the correct merchant/environment.
4. **Wait for propagation**
   Cashfree documents that supported UPI apps may take approximately 30-40 minutes to appear after enablement.
5. **Repeat the full matrix**
   Test Android, iOS, installed apps, no-app fallback, cancel, success, pending, and return flows.

Do not enable early	Cashfree explicitly warns that enabling the feature before native UPI handling is complete can break the checkout journey because the WebView will try to load UPI links as web pages.

7. QA plan and acceptance criteria

ID	Test	Expected result	Owner
A1	Open payment site in Android WebView	Pay-in form loads; no mixed-content or blank-page error	Android/QA
A2	Select GPay/PhonePe/Paytm	Installed native app opens; WebView does not show Web Page not available	Android/QA
A3	Cancel in UPI app	Return to same checkout; customer can recover or retry	Android/QA
A4	No supported app installed	Clear fallback; no crash or blank page	Android/QA
I1	Open payment site in iOS WKWebView	Pay-in form loads and remains stable	iOS/QA
I2	Select an installed supported UPI app	UIApplication opens the PSP and WKWebView cancels navigation	iOS/QA
I3	canOpenURL false	Clear fallback; no navigation dead end	iOS/QA
C1	Complete sandbox payment	Backend verification returns PAID before success UI	All
C2	Return with ACTIVE order	UI stays pending; no false success	All
C3	Refresh return URL	Same order can be verified idempotently	Web/QA
C4	Webhook signature failure	Backend rejects with 401	Backend/QA
S1	Inspect mobile binary/network	No Cashfree secrets are present	Security
S2	Try unlisted custom scheme	App does not launch arbitrary external handler	Security

Required test conditions

- Physical Android and iOS devices; emulators/simulators are not enough for the final UPI result.
- At least one relevant UPI app installed and SIM binding completed in that PSP app.
- Cashfree sandbox merchant and feature flag enabled only after handlers are ready.
- Final HTTPS payment origin configured in APP_BASE_URL and loaded by the mobile builds.
- Backend logs available, but no secrets or raw sensitive payloads copied into tickets.

Definition of done

Mobile acceptance	Both production-candidate builds launch supported UPI apps, handle missing apps and cancellation, preserve the checkout state, return to RealFrnd, and display success only after backend verification reports PAID.

8. Security, deployment, and go-live gates

Immediate actions before sharing with mobile

- Rotate all Cashfree PG and Payout credentials exposed during setup; do not reuse them.
- Set ENABLE_SANDBOX_PAYOUTS=false while the public POC has no authentication.
- Do not share .env, PM2 environment dumps, server logs containing secrets, or session IDs.
- Keep payout out of the UPI Intent mobile scope.

HTTPS migration

Current	Required before acceptance
http://13.127.207.172:5052	https://cashfree.<approved-realfrnd-domain>
Public port 5052	Nginx on 443 proxies privately to 127.0.0.1:5052
Browser shows Not Secure	Valid TLS certificate and no mixed content
IP-based APP_BASE_URL	HTTPS origin used for Cashfree return_url

Production gaps in the current POC

- Authenticate and authorize every order and payout endpoint.
- Derive amount and customer identity from trusted server data, not only browser form input.
- Persist orders, payment attempts, webhook event IDs, and reconciliation state in a database.
- Make webhook processing idempotent and store verified events before returning 200.
- Add CSRF controls, rate limiting, audit logs, monitoring, alerting, and secret management.
- Keep the app on a supported Node.js LTS release and patch dependencies.
- Restrict the mobile WebView to reviewed origins and documented external schemes.

Current webhook limitation The POC verifies the Cashfree webhook signature but only logs the event. It does not persist payment state. Do not treat it as production reconciliation until storage and idempotency are implemented.

9. Troubleshooting guide

Symptom	Likely cause	Fix
Web Page not available	WebView tried to render upi:// or app-specific scheme	Intercept the URL, launch native app, cancel navigation
No UPI apps shown	Feature flag not enabled, propagation pending, or PSP not SIM-bound	Complete handlers, request flag, wait 30-40 minutes, verify device binding
Android query returns empty	Android 11+ package visibility missing	Add manifest queries for UPI schemes
iOS canOpenURL is false	Scheme missing from LSApplicationQueriesSchemes	Add exact scheme to Info.plist and rebuild
Return URL invalid	APP_BASE_URL missing host	Set a syntactically valid public origin; final value must be HTTPS
Payment app returns but UI is stale	WebView recreated or order status not rechecked	Preserve WebView and verify order from backend
Checkout works on desktop only	Native scheme interception not implemented	Complete Android/iOS handlers and physical-device tests
EADDRINUSE	Another process owns port 5052	Inspect PM2/ss/lsof; run one app instance

Copy-paste ticket for Android and iOS

Mobile engineering ticket	Integrate the RealFrnd Cashfree JS checkout inside the existing native WebView. Load the approved PAYMENT_WEB_URL. Intercept Cashfree UPI and app-specific URL schemes, open them in installed native apps, and cancel WebView navigation. Android must implement shouldOverrideUrlLoading, intent:// parsing, ACTION_VIEW, and Android 11+ queries. iOS must implement WKNavigationDelegate, canOpenURL/open, and LSApplicationQueriesSchemes. Preserve the existing WebView while the PSP is open. Do not include Cashfree credentials in either app. Treat only backend order_status=PAID as success. Complete no-app, cancel, pending, success, and return-flow tests on physical devices before requesting the Cashfree UPI Intent feature flag.

Handoff checklist

Acceptance item	Sign-off
[ ] Final HTTPS PAYMENT_WEB_URL provided	
[ ] Android manifest queries merged	
[ ] Android URL interception merged	
[ ] iOS Info.plist schemes merged	
[ ] iOS WKNavigationDelegate interception merged	
[ ] No-app fallback approved by product/design	
[ ] Physical-device QA passed on both platforms	
[ ] Credentials absent from mobile builds	
[ ] Cashfree feature flag enabled after implementation	
[ ] Backend PAID verification demonstrated	

End of handoff